Course: Individual Software Development Process' 2026

Software Requirement Specification

By Wassawin the Goat of All Time (W-GOAT)

Chosen Irl Challenge: Project B: Lab Workflow & Request Management

Kantinan Lertluckpreecha	6810545476
Jedsada Ungsunantwiwat	6810545522
Pathadon Aougsk	6810545808
Wassawin Swangjaeng	6810545905
Siraphob Phonphakdee	6810545930

Table of Contents

1. Target Users

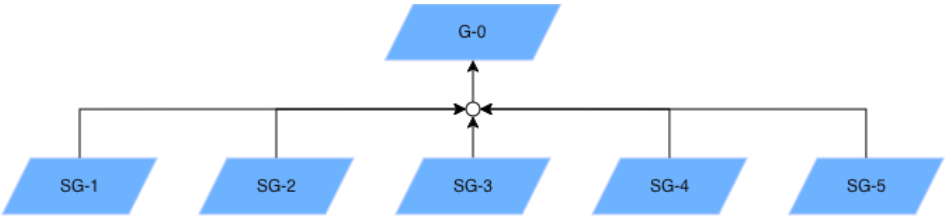
- **Lab Members:** View, create, and update tickets related to experiment setup, equipment maintenance, access permissions, consumables ordering, and small internal tickets.
- **Lab Staff (System administrator):** Maintain structured information, assigned task and process Lab members ticket.

  Clarification
  Requests are the focus of our app. It represents the work that has to be done by somebody inside the lab. There are two types of requests, which are:
  - **Tickets**. They are meant for members to ask for inquiries in the lab such as: a fix, equipment maintenance, setting up, or food orders to the administrator.
  - **Tasks**. They are meant for the administrator to assign work to the lab members that may or may not be related to the ticket.

2. The System Goal

G-0: Maintain efficient and accessible lab workflow and request management.
The lab currently relies on fragmented tools such as Discord, Line group chats, Gmail, and verbal communication to manage lab requests and workflow, causing delays, missed requests, and difficulty tracking the status of ongoing tasks. For this iteration, the system scope is limited to **centralized lab request submission, ticket tracking and workflow management, task and progress updates, request searching and history, announcements, and detailed reports**. **Lab Members** can create and update their tickets and view the assigned task. At the same time, **Lab Staff (System Administrator)** can assign tasks, judge tickets from lab members (Approve, Revised, Reject), and overall maintain structured information and administrative access.
Other functions such as inventory management and equipment management are explicitly out of scope for this iteration and will be addressed in later iterations.

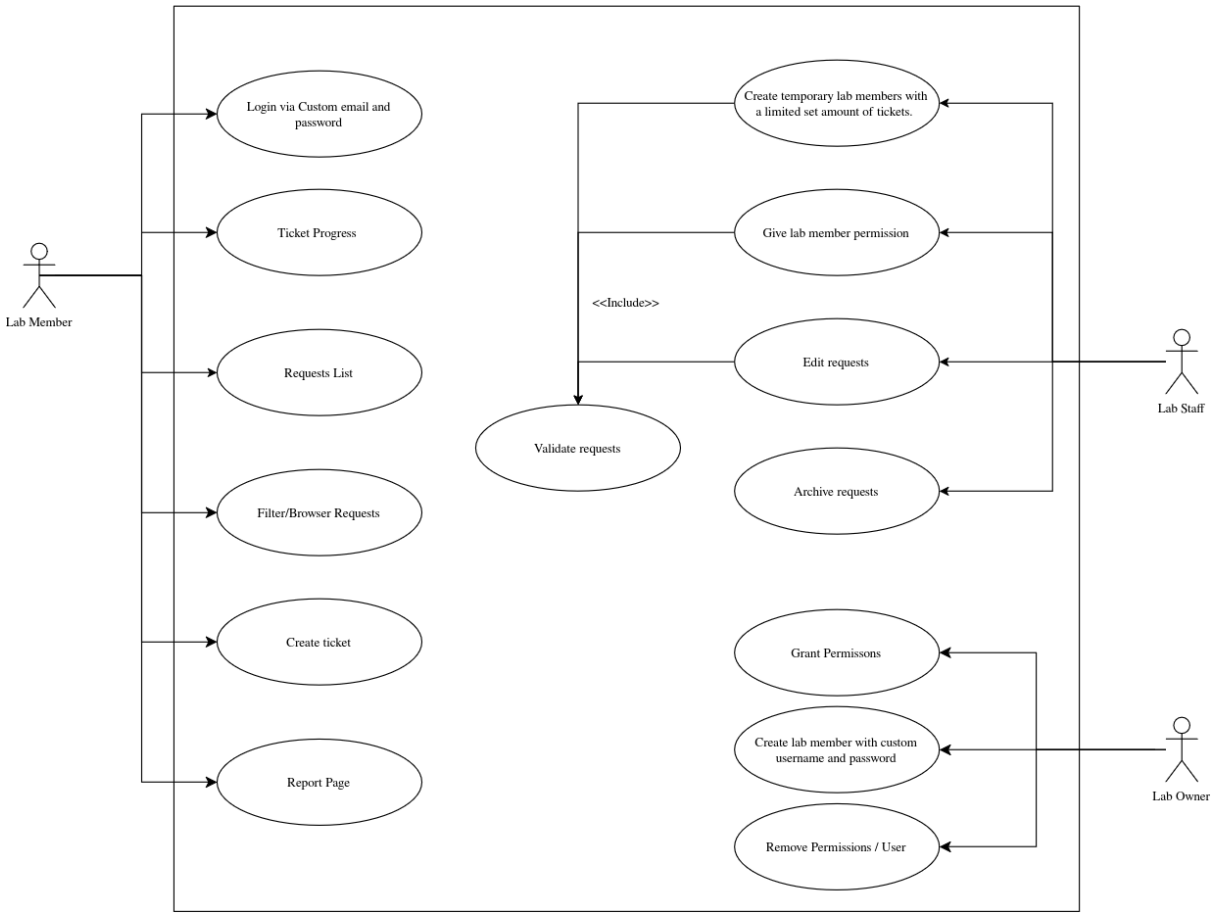
3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1)**
  Description: Achieve: Centralized submission of lab request and workflow items.
  Rationale: Requests are currently scattered across email, chat, and spreadsheets, which is why things get lost.

- **Sub-Goal 2 (SG-2)**
  Description: Maintain: Complete and consistent request records.
  Rationale: Each request needs a name, description, request by, approved by, priority, status, and timestamps so it can actually be tracked.
- **Sub-Goal 3 (SG-3)**
  Description: Achieve Clear ownership and follow-up tracking for every request.
  Rationale: Follow-up is hard to track today. Someone needs to be visibly accountable for each item.
- **Sub-Goal 4 (SG-4)**
  Description: Avoid: request being lost or left unaddressed without visibility.
  Rationale: Untracked requests undermine lab operations, delay experiments, and erode trust in the process.
- **Sub-Goal 5 (SG-5)**
  Description: Maintain Central, up-to-date visibility of lab activity and resource usage.
  Rationale: New lab members currently have no central place to see "what's going on" or how to request support.

4. Use Case Diagram



5. Use Stories

User Story ID	Description	Acceptance Criteria
US-1	As a Lab Member, I want to submit new lab tickets so that I can get support from the lab staff.	The Lab Member can submit a ticket by providing the required information such as title, description, category, and priority, and the system saves the request and generates a unique request ID.
US-2	As a Lab Member, I want to view and track my ticket so that I can know the current progress of my requests.	The Lab Member can view their submitted tickets and see information such as the request ID, title, priority, and current status, with the information updated when the tickets progresses.
US-3	As a Lab Member, I want to update the assigned task with additional information or status. So all people in the lab have the most accurate information.	The Lab Member can update the details of an assigned task, such as its status.
US-4	As a Lab Staff, I want to assign tasks to responsible staff members so that each request can be properly managed.	The Lab Staff can select an available staff member and assign them to a task, and the assigned staff member is displayed on the request.
US-5	As a Lab Staff, I want to update the status and progress of tickets so that Lab Members can know what is happening with their requests.	The Lab Staff can update the tickets status and add a progress note, and the system saves the changes so that the latest progress is visible to authorized lab members.
US-6	As a Lab Member, I want to search, filter, and view request history so that I can easily review previous lab activities.	The Lab Member can search and filter tickets/tasks using information such as request ID, category, priority, or status, and can view archived tickets/tasks as part of the request history.
US-7	As a Lab Staff, I want to create new categories for putting requests in relevant topics. So it is easier for me to sort through requests and for better organization.	The Lab Staff can create new categories that allow requests to be classified.
US-8	As a lab staff I want other people that are not in the lab to be able to create tickets based on the limit I set for each individual.	The Lab staff can grant a limited amount of tickets to users that are not related to the lab.

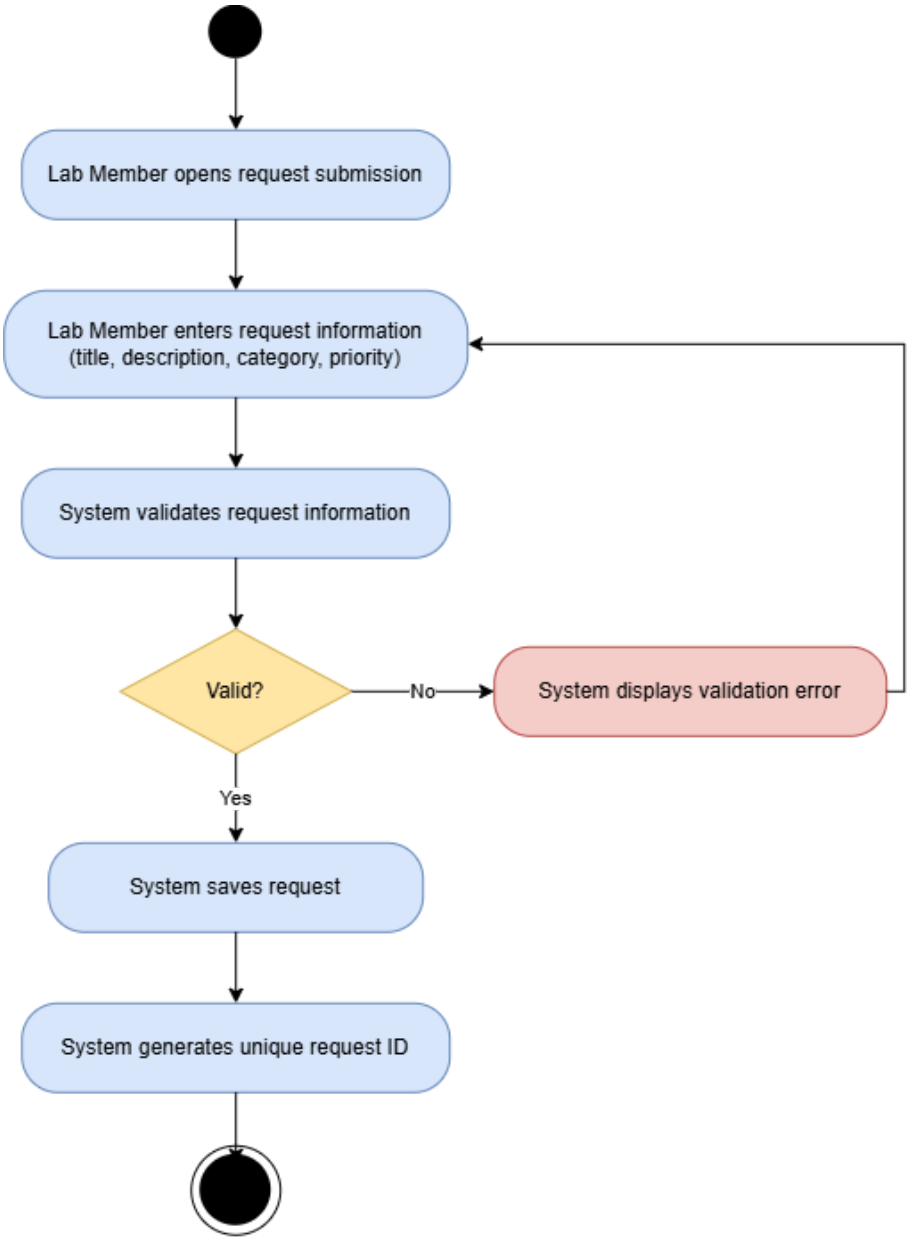
6. User Requirement Specification

ID	User Requirement
URS-1	The system shall allow lab members to submit new lab tickets.
URS-2	The system shall allow lab members to view and track their tickets.
URS-3	The system shall allow selected lab members to edit and update some lab tasks.
URS-4	The system shall allow lab staff to assign and manage tasks.
URS-5	The system shall allow authorized lab members to update lab tickets status and tasks progress.
URS-6	The system shall allow lab members to search, filter, and view lab log history.
URS-7	The system shall allow lab staff to create new categories for other members to assign requests in relevant categories.
URS-8	The system shall allow lab staff to grant a limited amount of tickets to users that are not in the lab.

7. Activity Diagram

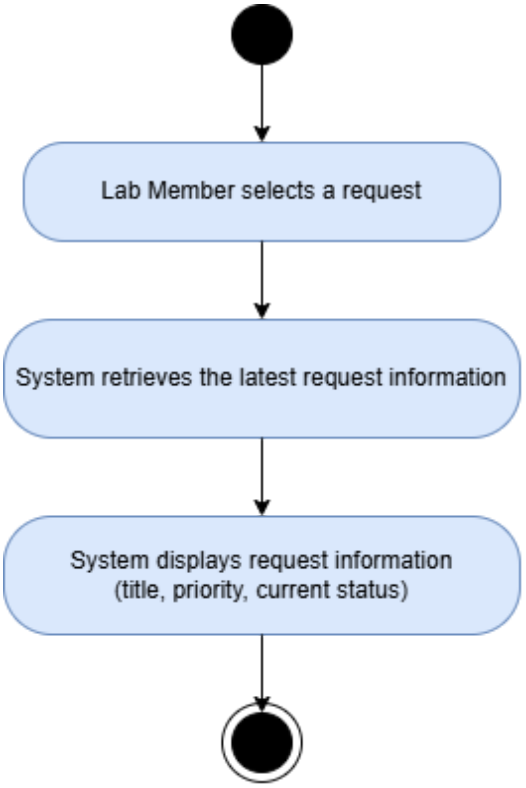
AD-1: Lab Member creates a new request

This diagram traces the lab-member request submission flow used to derive SRS-1.



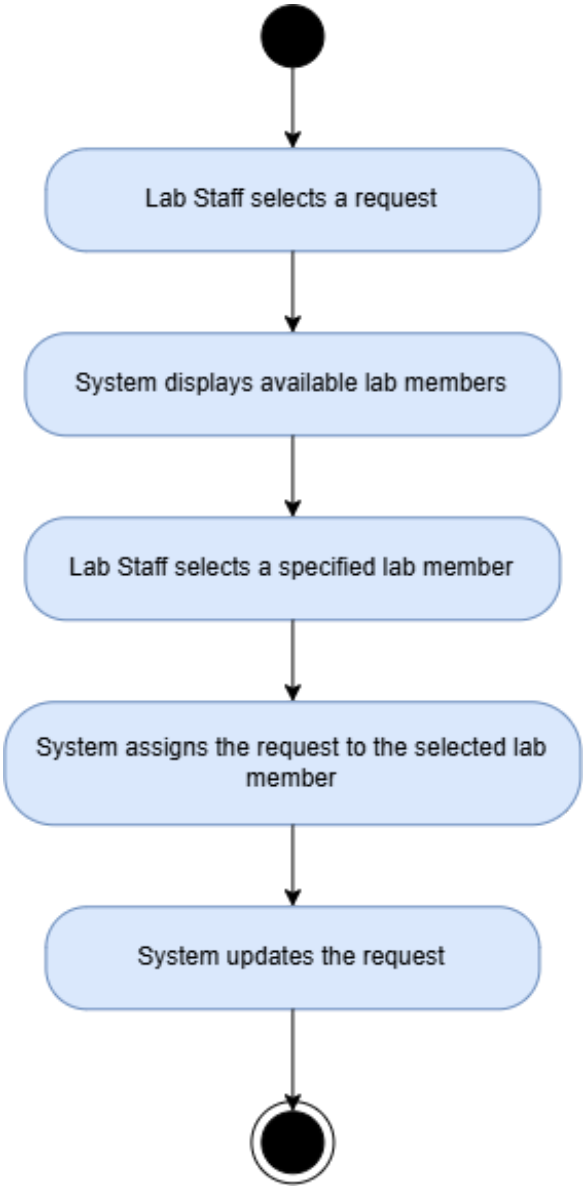
AD-2: Lab Member views and tracks a requests

This diagram traces the lab-member request viewing and tracking flow used to derive SRS-2.



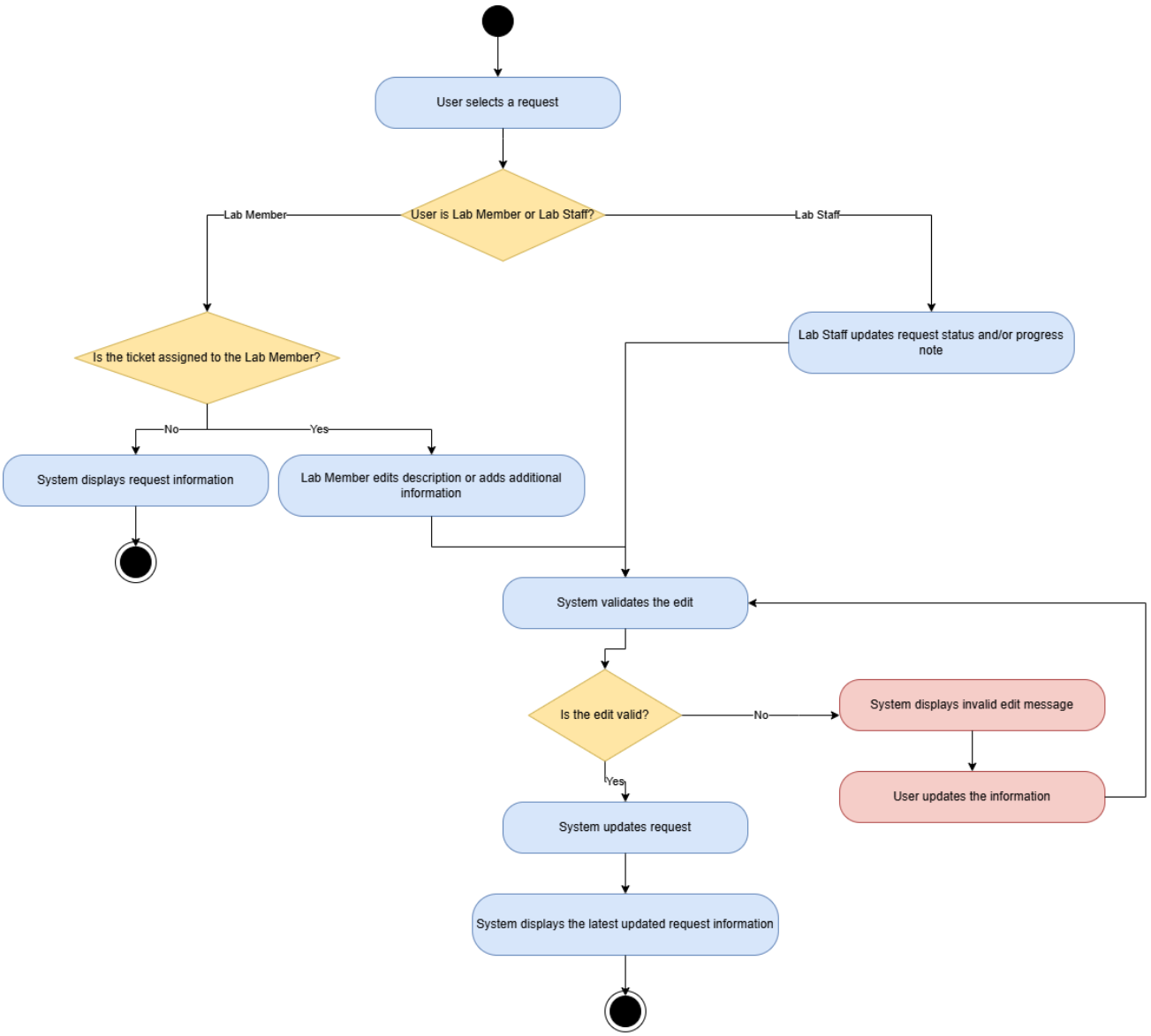
AD-3: Lab Staff assigns a requests

This diagram captures the request assignment flow used to derive SRS-4.



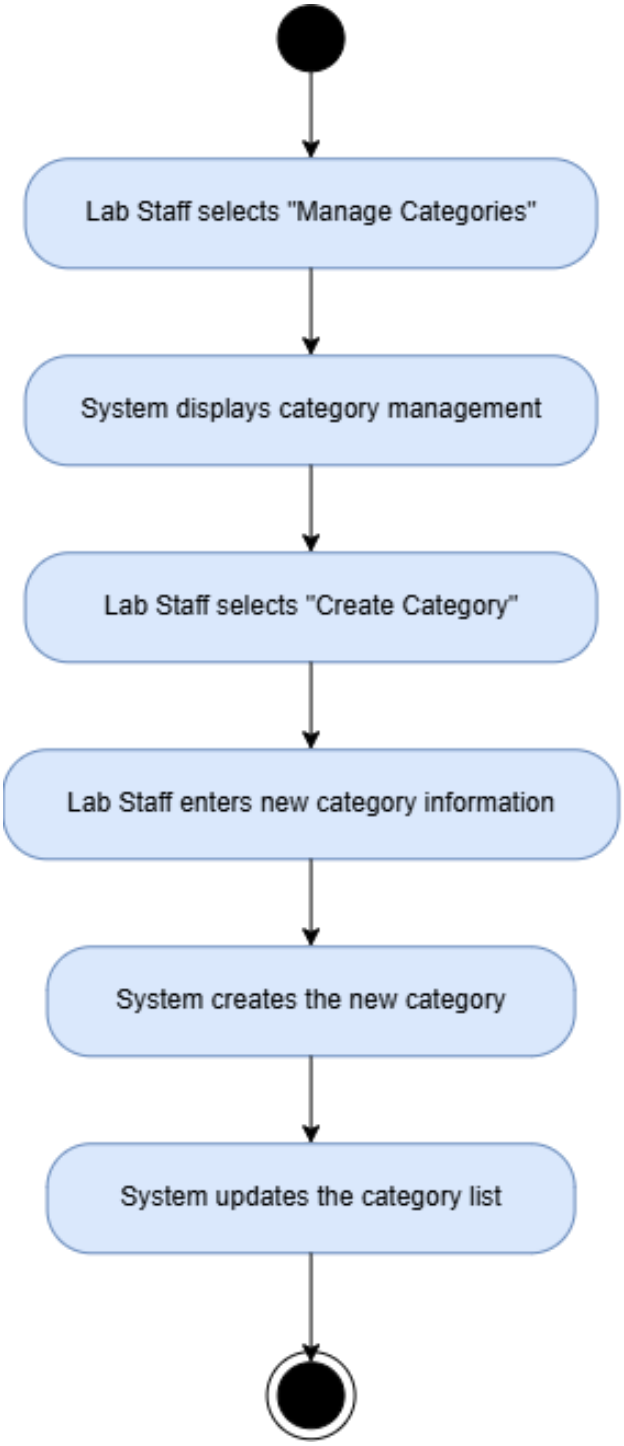
AD-4: Lab Staff updates and Lab Member requests information

This diagram captures the request update flow used to derive SRS-3, SRS-5, and SRS-7.



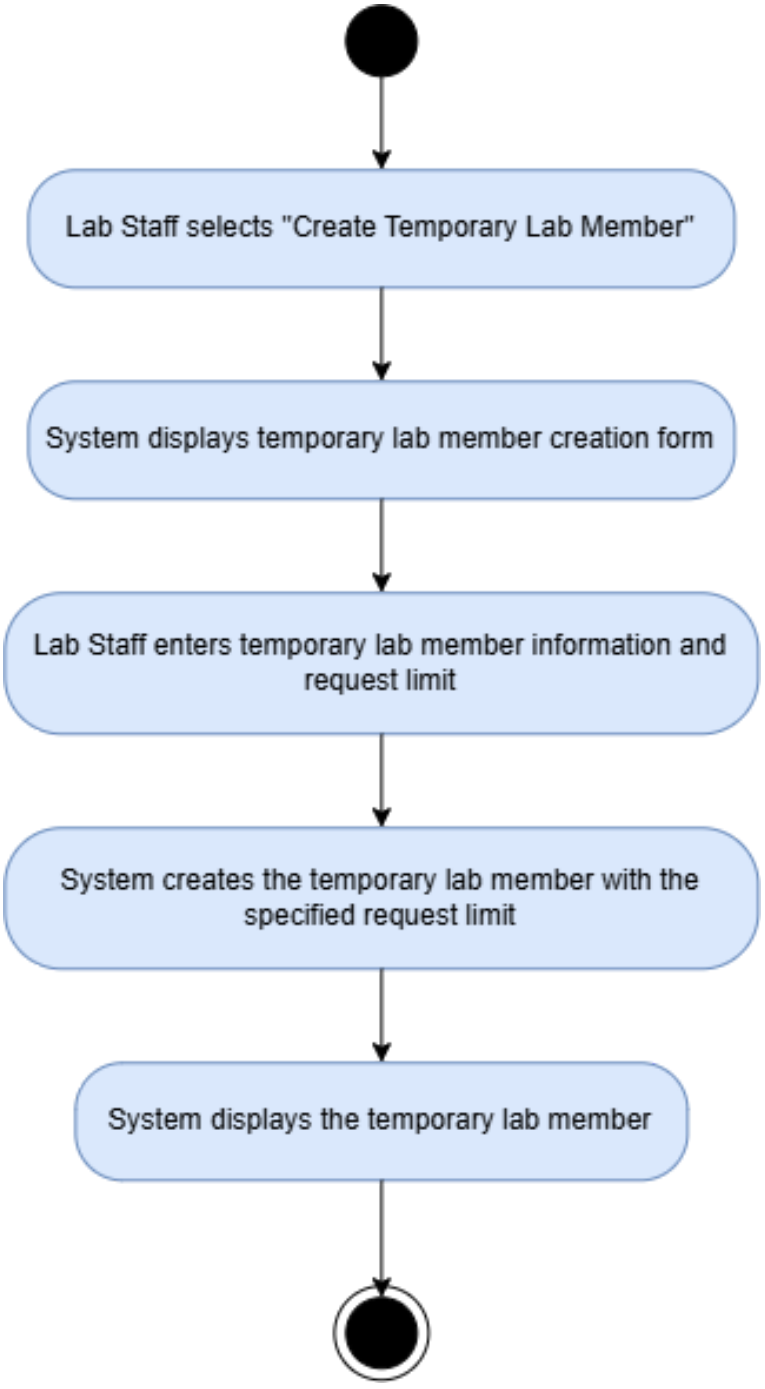
AD-5: Lab Staff creates a new category

This diagram captures the category creation flow used to derive SRS-8.



AD-6: Lab Staff creates a temporary lab member

This diagram captures the temporary lab member creation flow used to derive SRS-9.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall allow lab members to create new tickets in the allowed categories.
SRS-2	The system shall display the title, ticket priority, and current status with the latest updated information.
SRS-3	The system shall allow lab members to edit the description or add additional information of an assigned task.
SRS-4	The system shall allow lab staff to be able to assign tasks to specified lab members.
SRS-5	The system shall allow the lab staff to be able to update the tickets status and progress note accordingly.
SRS-6	The system shall allow members to search, filter, and view log history.
SRS-7	The system shall be able to update each request after every valid edit or creation.
SRS-8	The system shall allow lab staff to create new categories based on lab staff's needs.
SRS-9	The system shall allow lab staff to create temporary lab members with a limited set amount of tickets.

9. Software Architecture

Based on the current system scope for centralized lab request submission, workflow management, tracking, assignment, status updates, searching, and administrative access, the system adopts a Monolithic Architecture. The application is developed and deployed as a single unit with a shared codebase and database, where all system functions operate within the same application.

Rationale

The system supports the core lab workflow in US-1 to US-6: creating requests, viewing and tracking requests, editing tickets information, assigning tasks, updating status and progress, and searching, filtering, and viewing log history. To achieve this a Monolithic Architecture is suitable because the current system is within a limited scope and can be developed, tested, and deployed as a single unit without the operational complexity of independently deployed services. Since the project does not require independent scaling of individual features, separating these functions into microservices would add unnecessary complexity.The shared application and database also keep communication and data management straightforward for the current system. Although individual features cannot be scaled independently and the application has a single deployment unit, this trade-off is acceptable for the current scope and development stage.

Component	Layer/Type	Description
Web Client	Presentation (View)	Serves as the user interface for Lab Members, Lab Staff, System Administrators, and temporary lab members; displays tickets/tasks submission forms, tickets/tasks status-tracking dashboards, search filters, and progress notes, and consumes responses from the Backend Controller.
Backend Controller	Controller	Receives incoming HTTP requests for tickets/tasks actions (create tickets/tasks, update status, assign handler, create category, grant temporary user quota), executes cross-cutting authorization checks, and orchestrates calls to the tickets/tasks Logic layer before returning responses to the View.
Requests Logic	Model	Encapsulates core business rules for tickets/task management: validates required fields (title, description, category, priority), generates unique ticket IDs, manages tickets/task status transitions, enforces non-lab user request limits, and handles search/filter logic.

Requests Database	Persistence	Stores structured system records, including requests details, user information, assignment history, progress notes, request categories, and temporary access limits execute queries for active requests, tracking views, and request history reports.
Authentication/Authorisation Module	Cross-cutting	Verifies identity and enforces role-based access control (Lab Member, Lab Staff, System Administrator) before allowing restricted actions such as assigning handlers, creating categories, or granting tickets quotas.

10. Data Storage Technology

For this iteration of the system, a Relational Database Management System (RDBMS) like PostgreSQL which will be hosted on supabase will serve as our primary data storage solution.

Rationale

The core functionality of our lab request system relies heavily on structured data with clear, interconnected relationships. Based on our design, entities such as lab members, tickets, categories, and task assignments map perfectly to a relational model. An RDBMS is ideal here because it enforces strict data integrity through foreign keys, for instance, ensuring that a task is only assigned to a valid lab member (SRS-4) and is categorized correctly (SRS-8). Furthermore, tracking lab requests requires reliable, fail-safe state changes. When a staff member updates a ticket status (SRS-5), we need transactional consistency to ensure that both the main ticket table, task table and the audit logs are updated simultaneously without data corruption (SRS-7). A relational database provides these transactional guarantees natively. Since our current data scope is well-defined and predictable, introducing a NoSQL database right now would just add unnecessary operational overhead.

Component	Data Storage Technology	Description
Lab Management Database	RDBMS (PostgreSQL on Supabase)	A unified relational database containing normalized tables. It handles all data persistence, relational mapping, and integrity constraints needed to track the lab's workflow.

Currently, a single relational database can fully handle our structured ticketing and user management requirements. However, the current Supabase plan is limited to 500 MB for overall database system. Storing more than this would require additional cloud storage costs, which can become relatively expensive for the project. Therefore, the system should manage its storage usage within this limit during the current iteration. In future iterations, if the lab needs to expand the system to handle larger amounts of data or unstructured data such as large experimental data files, flexible equipment manuals, or real-time sensor streams, additional storage capacity or a supplementary NoSQL store may be considered without rebuilding the core ticketing system.

11. Development Environment

For our deployment environment we decided to go with dockers to improve, unify and streamline the process of development experience of the group.

Rationale

The system uses Docker containers for development and deployment of the application. The Web Client and backend application (Backend Controller and ticket logic) each run in a container defined in a shared Docker Compose configuration. So every team member builds and runs exactly the same environment regardless of their host machine. The database does not contain both local development and production connected to Supabase, so the team never had to install or maintain postgresql itself, and backups, upgrades and storage are handled by the provider. This does remove the "Work on my machine" problems. The backend remains a single monolithic application. The web client is a separate web application that communicates over HTTP. The current lab workload is small and predictable enough that the overhead of containers are acceptable. Even though we need to maintain the docker configuration and that development needs a network access to superbase rather than a fully offline environment.

Component	Development Environment	Description
Web Client	Docker	Runs the web application in its own container and provides the interface for Lab Members, Lab Staff, System Administrators, and temporary lab members it communicates with the Backend Controller over HTTP.
Backend Controller	Docker	Runs in the backend application container and handles HTTP requests, authorization checks, requests processing, assignment, status updates, category management, and temporary-user quota operations.

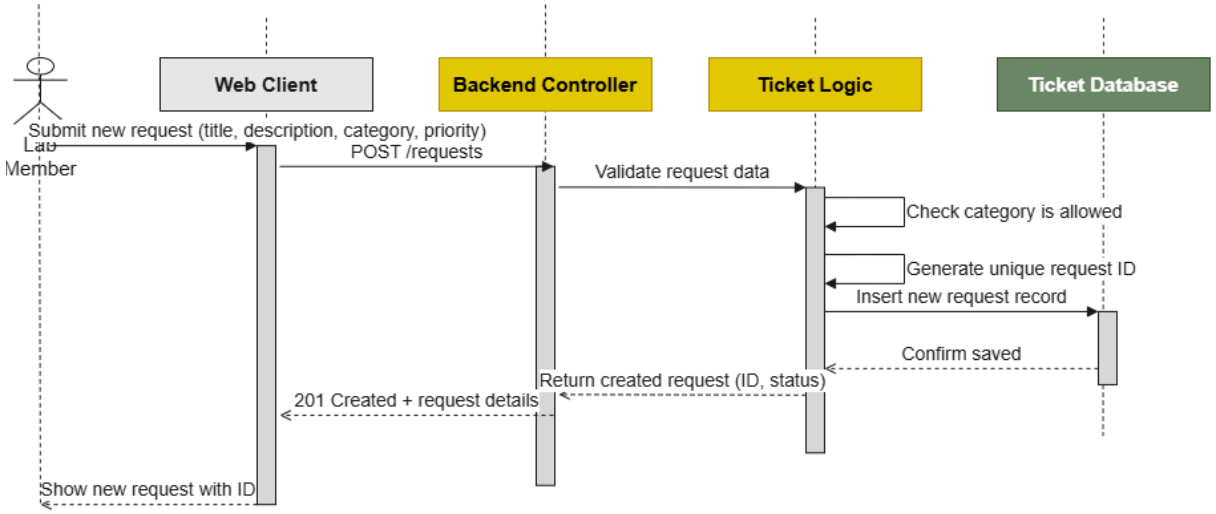
Request Logic	Docker	Runs as part of the backend application directly on the host machine and implements business rules such as request validation, ticket ID generation, status transitions, search/filter operations, and requests limits.
Lab Database	Supabase (cloud)	Used by the development team to build, test, and run the Web Client and backend containers with Docker Compose, connected to Supabase; only Docker and the source code need to be installed on the machine.
Local Developmen t Machine	Physical Machine	Used by the development team to build, test, and run the system directly on a physical computer with the required application runtime, dependencies, and PostgreSQL installation.
Production Environment	Cloud	Cloud host that runs the Web Client and backend containers and connects to the Supabase database, providing the centralized environment used by lab members and staff.

12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the chosen Monolithic Architecture (Section 9) and cover all system requirements defined in Section 8. The diagrams below collectively trace SRS-1 through SRS-9.

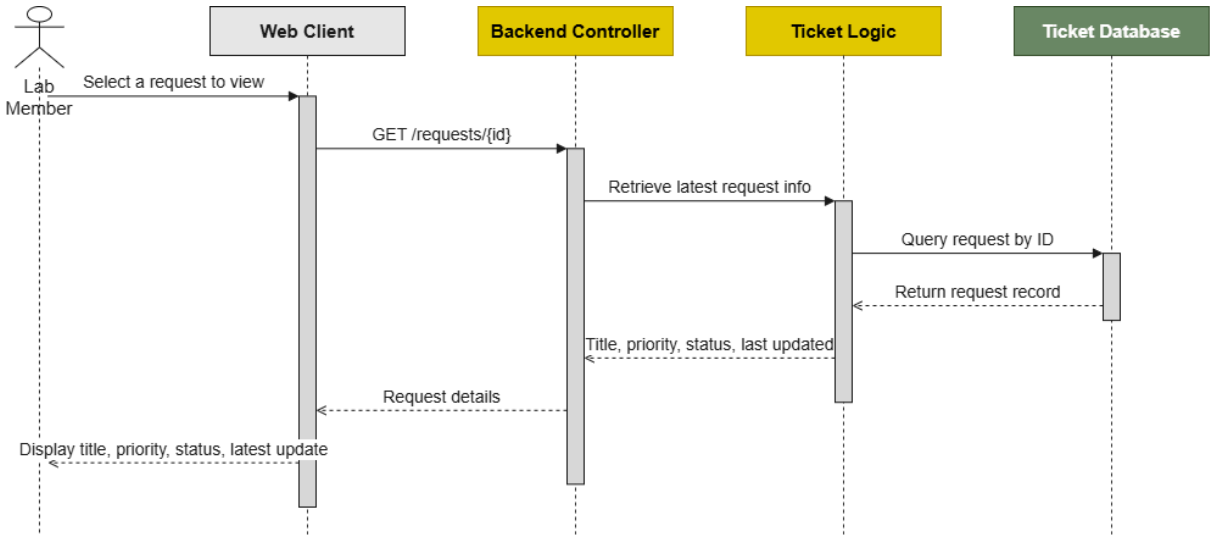
SQD-1: Lab Member Creates a New requests

Traced requirements: SRS-1, SRS-7



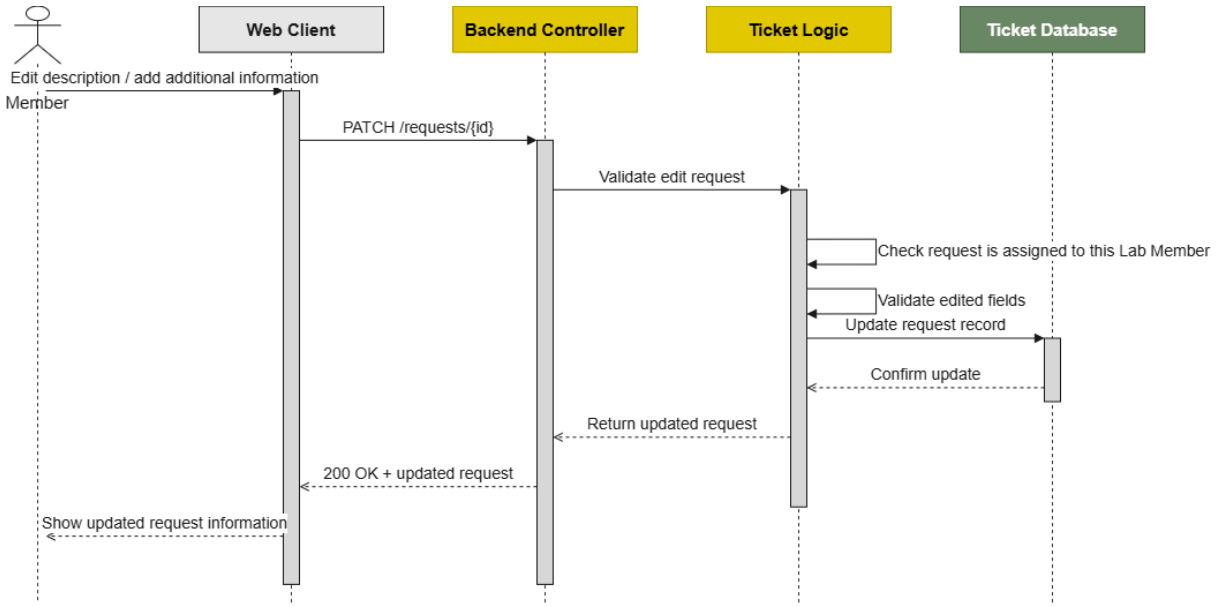
SQD-2: Lab Member Views and Tracks a requests

Traced requirements: SRS-2



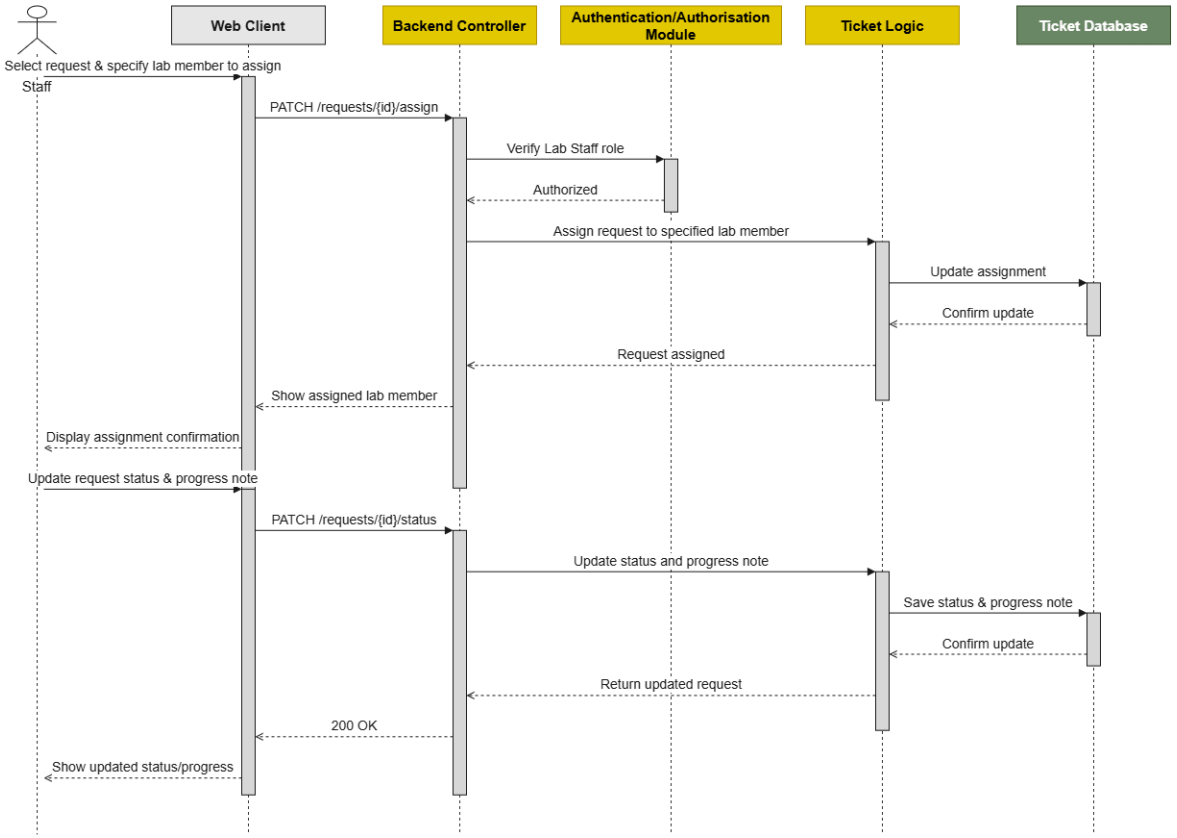
SQD-3: Lab Member Edits requests Information

Traced requirements: SRS-3, SRS-7



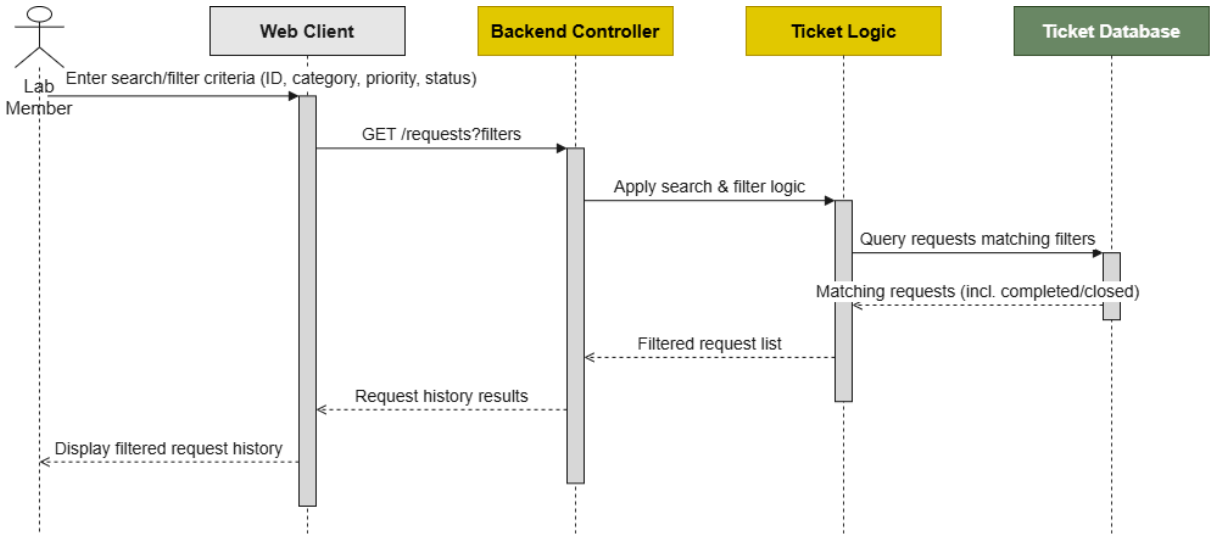
SQD-4: Lab Staff Assigns and Updates a requests

Traced requirements: SRS-4, SRS-5, SRS-7



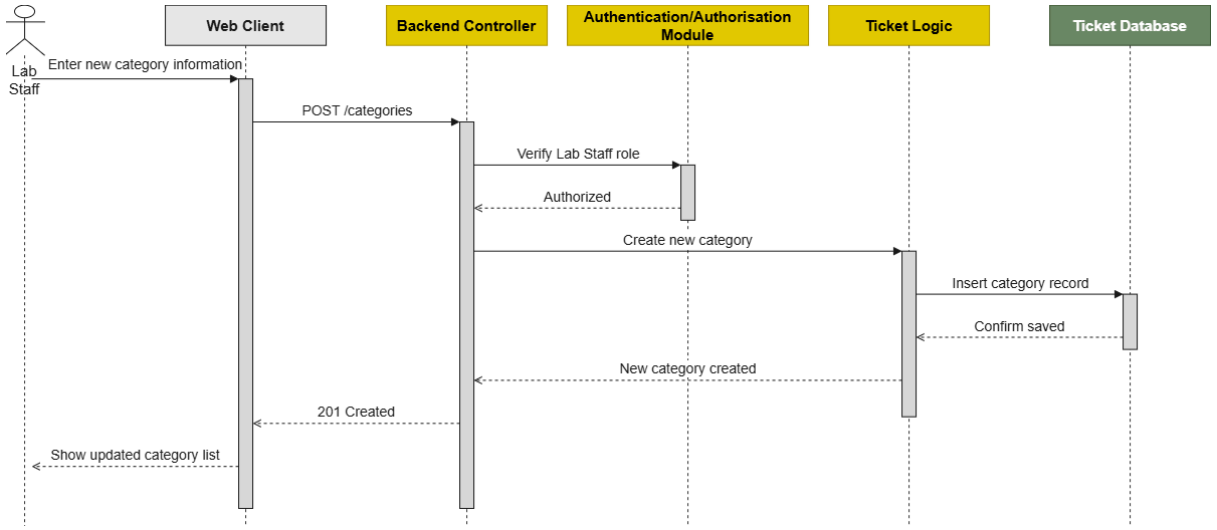
SQD-5: Lab Member Searches, Filters, and Views History logs

Traced requirements: SRS-6



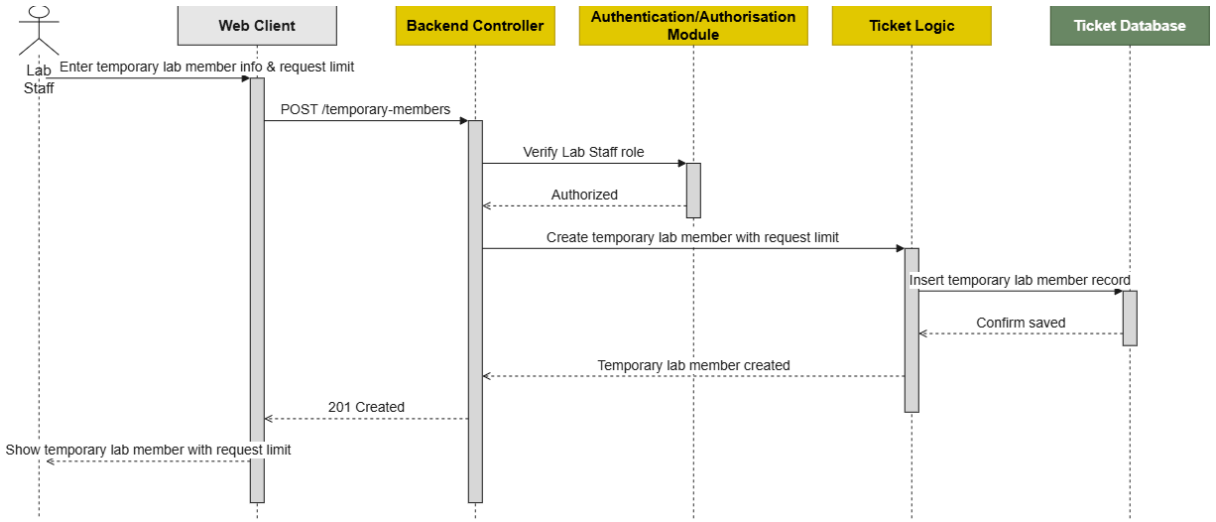
SQD-6: Lab Staff Creates a New Request Category

Traced requirements: SRS-8



SQD-7: Lab Staff Creates a Temporary Lab Member

Traced requirements: SRS-9



13. Traceability Matrix

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 Create ticket	US-1	URS-1	AD-1	SRS-1
SG-2	UC-1 Create ticket, UC-2 Ticket Progress, UC-7 Edit ticket response, UC-9 Validate ticket	US-1, US-2, US-3, US-5	URS-1, URS-2, URS-3, URS-5	AD-1, AD-2, AD-4	SRS-1, SRS-2, SRS-3, SRS-5, SRS-7
SG-3	UC-2 Ticket Progress, UC-6 Give lab member permission, UC-7 Edit ticket response, UC-10 Create temporary lab member	US-2, US-3, US-4, US-5, US-8	URS-2, URS-3, URS-4, URS-5, URS-8	AD-2, AD-3, AD-4, AD-6	SRS-2, SRS-3, SRS-4, SRS-5, SRS-7, SRS-9
SG-4	UC-2 Ticket Progress, UC-3 Ticket List, UC-4 Filter/Browse ticket, UC-8 Archive ticket	US-2, US-5, US-6	URS-2, URS-5, URS-6	AD-2, AD-4	SRS-2, SRS-5, SRS-6, SRS-7
SG-5	UC-3 Ticket List, UC-4 Filter/Browse ticket, UC-5 Report Page, UC-11 Create Category	US-2, US-6, US-7	URS-2, URS-6, URS-7	AD-2, AD-5	SRS-2, SRS-6, SRS-8

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-1
SRS-2	SQD-2
SRS-3	SQD-3
SRS-4	SQD-4
SRS-5	SQD-4
SRS-6	SQD-5
SRS-7	SQD-1, SQD-3, SQD-4
SRS-8	SQD-6
SRS-9	SQD-7